

PROJECT PROPOSAL

Solving and Generating Sudoku as a Constraint Satisfaction Problem

Group Number: 3

Date: 07/21/2026

Team Members:

- Arjun Mohunta, amohunta, 80654131
- Haoyang Ding, haoyad6, 44024972
- Rajat Choudhary, rajatc1, 37222868

1. Main Idea and Problem Definition

Our project builds a system that treats Sudoku as a formal reasoning problem rather than a game of chance or trial-and-error. The system has two goals from the end user's perspective. First, given any valid, partially filled 9×9 Sudoku grid, it returns the unique completed solution. Second, it generates brand-new puzzles at a requested difficulty level, each guaranteed to have exactly one solution. A user should be able to hand the system a grid and get a correct answer in a fraction of a second, or ask for a fresh puzzle of a chosen difficulty and receive a well-formed one.

Formally, Sudoku is a constraint satisfaction problem: there are 81 variables, each with a domain of 1-9, subject to "all-different" constraints across every row, every column, and every 3×3 box. Consistent with the course's framing of puzzles as tasks with no feedback from an environment, success comes entirely from search and logical inference over these constraints rather than from reinforcement-style feedback. This proposal focuses on the problem setup rather than committing to a single implementation detail.

2. Input and Output

Input

- Solver: a 9×9 grid supplied as a text file or an 81-character string, with 0 or a blank marking empty cells. Optionally read from a simple grid UI.
- Generator: a target difficulty level and an optional random seed for reproducibility.

Output

- Solver: the completed 9×9 grid, a validity flag, and solver statistics such as run time, number of backtracks, and search nodes expanded.
- Generator: a new puzzle with its solution and an estimated difficulty rating, guaranteed to admit exactly one solution.

Applications

The same techniques apply to real problems like scheduling, timetabling, and resource assignment, so the solver is a small testbed for methods that reach beyond Sudoku. It also works as a teaching tool for showing how constraint reasoning narrows a large search space.

3. Basic Approach

We will model the Sudoku grid as a constraint satisfaction problem. 81 cells, each holding a value from 1-9, with the rule that no number repeats in any row, column, or 3×3 box.

Our main solver uses backtracking search with constraint propagation: it fills cells one at a time, and after each choice it removes that value from the candidates of related cells. To search efficiently, it always fills the cell with the fewest remaining options first. If a cell runs out of valid options, it backtracks and tries a different value.

As a comparison point, we will also implement a plain backtracking solver as a baseline, so we can measure how much the smarter techniques improve speed and reduce guesswork.

The generator reuses the solver to make new puzzles. It starts with a full, valid solved grid, then removes numbers one at a time. After each removal, it runs the solver to check the puzzle still has only one solution, and undoes any removal that creates more than one. Difficulty is set by how many numbers are left and how hard the required solving techniques are, which also gives us a way to rate each puzzle's difficulty.

4. Evaluation Plan

Quantitative

- Metrics: solve rate (percentage of puzzles solved correctly), average solve time, and number of backtracks and search steps used.
- Baseline: plain backtracking is our baseline. We compare it against our main solver to measure how much constraint propagation and the Minimum Remaining Values heuristic reduce solve time and backtracking. On hard puzzles, we expect roughly a 10 to 100x reduction in backtracks and a comparable drop in solve time, turning cases that take a plain solver millions of search nodes into near-instant solves.
- Data: we will test on public Sudoku datasets covering a range of difficulty levels, primarily the Kaggle "1 million Sudoku games" dataset for broad solve-rate and timing statistics, supplemented by the well-known "top 95" hard puzzles and individual hard cases such as AI Escargot to stress-test the solver on puzzles engineered to maximize backtracking.

Qualitative

- Sanity checks: an already-solved grid returns instantly; an easy puzzle solves with zero backtracks; an unsolvable or invalid grid is correctly flagged.

- Visualization: show the solver filling the grid step by step and highlight where it backtracks, so we can confirm the search behaves as expected.
- Moonshot: solve the hardest known Sudoku puzzles in well under a second.

5. Project Milestones

Milestone 1 - Basic design (Weeks 1 to 2): Decide how to store the Sudoku grid in the program and how to read puzzles in and print answers out, gather some test puzzles, and build a simple solver that fills cells one by one and undoes wrong guesses. The deliverable is a working version that solves easy and medium puzzles.

Milestone 2 - Core solver (Weeks 3 to 4): Make the solver smarter so it rules out impossible numbers as it goes and always fills the most restricted cell first. The deliverable is an improved solver that handles hard puzzles quickly.

Milestone 3 - Testing and metrics (Week 5): Build a setup that measures how fast the solver runs, how many wrong guesses it makes, and how many puzzles it gets right, then compare the smart solver against the simple one.

Milestone 4 - Visualization (Week 6): Show the solver working step by step so we can watch it fill the grid and see where it undoes guesses, to confirm it works as expected.

Milestone 5 - Final version (Week 7): Test the solver on all the puzzles, then write up the results and prepare the demonstration.